

UNIVERSIDAD DE GRANADA

Doble Grado de Ingeniería Informática y Administración y Dirección de
Empresas

Práctica 1: Técnicas de Búsqueda de Poblaciones para el Problema de Portfolio

Asignatura: Metaheurística

Curso Académico: Curso 2025-26

Estudiante: Ismael Sallami Moreno

DNI: 20888108Z

Email: ism350zsallami@correo.ugr.es

Horario de Prácticas: Jueves 15:30 - 17:30

1 de abril de 2026

Índice

1. Descripción y Formulación del Problema	2
1.1. Función Objetivo (Fitness)	2
1.2. Casos de Estudio	2
2. Aplicación de los Algoritmos Empleados al Problema	3
2.1. Esquema de Representación de las Soluciones	3
2.2. Evaluación de la Función Objetivo	3
2.3. Operador Común de Reparación de Soluciones	4
3. Estructura de los Métodos de Búsqueda	7
3.1. Búsqueda Aleatoria (Random Search)	7
3.2. Búsqueda Voraz (Greedy Search)	7
3.3. Búsqueda Local (Local Search)	9
4. Estructura del Código y Manual de Usuario	11
4.1. Estructura del Directorio <code>software/</code>	11
4.2. Manual de Usuario: Compilación y Ejecución	12
5. Experimentos y Análisis de Resultados	13
5.1. Descripción de los Casos de Estudio y Parámetros	13
5.2. Resultados Obtenidos	13
5.3. Análisis Crítico de Resultados	14
6. Trabajo Extra Voluntario	17
6.1. Búsqueda Local del Mejor frente al Primer Mejor	17
6.1.1. Descripción y Pseudocódigo	17
6.1.2. Resultados Obtenidos	18
6.1.3. Análisis de los Resultados	19
6.2. Búsqueda Local Multiarranque (Multi-start)	19
6.2.1. Descripción y Pseudocódigo	19
6.2.2. Resultados Obtenidos y Análisis Crítico	20
6.3. Análisis Visual Ampliado mediante Boxplots Mejorados	22
6.4. Arquitectura de Configuración Dinámica	24
Referencias Bibliográficas	25

1. Descripción y Formulación del Problema

El Problema del Portfolio (basado en el modelo de Markowitz) busca determinar la asignación óptima de capital entre N activos financieros para maximizar la rentabilidad esperada y minimizar el riesgo. La solución se codifica como un vector de pesos $w = (w_1, \dots, w_N)$, donde cada w_i representa la proporción del presupuesto invertida en el activo i . Toda cartera válida debe satisfacer estrictamente dos restricciones:

1. **Presupuesto completo:** $\sum_{i=1}^N w_i = 1$. Computacionalmente, se admite una tolerancia de error de 10^{-8} frente a la imprecisión de la coma flotante.
2. **Límites de inversión:** $w_i \in \{0\} \cup [lo, hi], \forall i$. El peso debe ser cero, o estar acotado en el rango legal permitido.

1.1. Función Objetivo (Fitness)

Se plantea un problema de optimización que combina linealmente el beneficio y el riesgo mediante un factor de penalización $\lambda = 500$. La función a **maximizar** se compacta en la Ecuación 1:

$$fitness(w) = \underbrace{\sum_{i=1}^N w_i \sum_{t=1}^T \log(1 + \mu_{it})}_{\text{Beneficio}} - \lambda \cdot \underbrace{\sum_{i=1}^N \sum_{j=1}^N w_i \cdot \Sigma_{ij} \cdot w_j}_{\text{Riesgo}} \quad (1)$$

Donde μ_{it} es el retorno relativo del activo i en el día t , y Σ_{ij} la covarianza histórica entre los activos i y j . El beneficio acumulado emplea logaritmos para simplificar el cálculo y estabilizar variaciones negativas.

Nota de diseño sobre el riesgo: El modelo penaliza el riesgo utilizando la *varianza* de la cartera ($w^T \Sigma w$) en lugar de su desviación típica (volatilidad). Esta es una decisión de diseño intencionada que mantiene la función objetivo como un problema cuadrático puro, evitando el alto coste computacional de ejecutar la operación de raíz cuadrada en cada evaluación de la función *fitness*. Puesto que la raíz cuadrada es una función monótona creciente, esta aproximación no altera la dirección de la optimización.

1.2. Casos de Estudio

La experimentación emplea datos históricos reales del periodo 2015-2025 para tres índices bursátiles:

- **IBEX 35** ($N = 30$): Bolsa española. Límites permitidos: $lo = 0,005$, $hi = 0,08$.
- **S&P 100** ($N = 97$): Principales empresas EE.UU. Límites permitidos: $lo = 0,005$, $hi = 0,05$.
- **S&P 500** ($N = 457$): Empresas EE.UU. Límites permitidos: $lo = 0,005$, $hi = 0,02$.

2. Aplicación de los Algoritmos Empleados al Problema

Antes de abordar la lógica interna de cada metaheurística particular, es imperativo definir la arquitectura común que subyace a todos los algoritmos desarrollados. Esta sección detalla cómo se mapea el problema financiero al dominio computacional, estableciendo la representación del espacio de búsqueda, la evaluación de la función objetivo y el mecanismo genérico de reparación de soluciones.

2.1. Esquema de Representación de las Soluciones

Dado un mercado con N activos financieros, cualquier solución candidata se representa computacionalmente como un vector unidimensional de números reales en doble precisión: $w = [w_1, w_2, \dots, w_N]$.

Cada componente w_i codifica la fracción del presupuesto total que se asignará a la empresa i -ésima. Esta codificación continua exige un riguroso control de validez en cada iteración de los algoritmos para garantizar que:

1. La suma total del vector converja a 1,0 (aplicando una tolerancia computacional $\epsilon = 10^{-8}$ para absorber la imprecisión de la aritmética de coma flotante).
2. Cada peso individual respete los umbrales legales del mercado: $w_i = 0$ (exclusión del activo) o $w_i \in [lo, hi]$.

2.2. Evaluación de la Función Objetivo

La función objetivo es transversal a todos los algoritmos. Dado que el beneficio acumulado se ha precalculado logarítmicamente durante la carga de datos para optimizar el rendimiento, la evaluación de cualquier cartera w se realiza mediante el Algoritmo 1.

Algoritmo 1: Evaluación de la Función Objetivo

Input: Vector de pesos w de tamaño N , Vector de beneficios acumulados B ,

Matriz de covarianzas Σ , factor λ

Output: Valor real de la función de evaluación (Fitness)

beneficio $\leftarrow 0,0$;

riesgo $\leftarrow 0,0$;

// Cálculo del beneficio esperado

for $i \leftarrow 1$ **to** N **do**

 | *beneficio* $\leftarrow$ *beneficio* + ($w_i \cdot B_i$);

end

// Cálculo del riesgo (Varianza de la cartera)

for $i \leftarrow 1$ **to** N **do**

for $j \leftarrow 1$ **to** N **do**

 | *riesgo* $\leftarrow$ *riesgo* + ($w_i \cdot w_j \cdot \Sigma_{ij}$);

end

end

return *beneficio* - ($\lambda \cdot$ *riesgo*);

2.3. Operador Común de Reparación de Soluciones

La exploración del espacio de búsqueda (ya sea mediante mutaciones estocásticas o asignaciones voraces) genera invariablemente vectores que violan las restricciones del problema. Para proyectar estas soluciones inválidas de vuelta al espacio factible, se ha diseñado un operador de reparación determinista, denotado como $\text{fix}(\mathbf{w})$.

Este operador actúa en tres fases: primero, una normalización base; segundo, una poda de límites; y tercero, un bucle de redistribución de excesos o déficits de capital. El Algoritmo 2 describe formalmente este procedimiento.

Algoritmo 2: Operador de Reparación $\text{fix}(w)$

Input: Vector de pesos inválido w , límites lo y hi , tolerancia ϵ
Output: Vector de pesos w factible

// Fase 1: Normalización inicial
 $suma \leftarrow \sum_{i=1}^N w_i$;
if $suma > 0$ **y** $|suma - 1,0| > \epsilon$ **then**
 | $w_i \leftarrow w_i / suma \quad \forall i$;
end
// Fase 2: Poda estricta de límites
for $i \leftarrow 1$ **to** N **do**
 | **if** $w_i < lo$ **then**
 | | $w_i \leftarrow 0,0$
 | **end**
 | **else if** $w_i > hi$ **then**
 | | $w_i \leftarrow hi$
 | **end**
end
// Fase 3: Redistribución iterativa
 $suma \leftarrow \sum_{i=1}^N w_i$;
while $|suma - 1,0| > \epsilon$ **do**
 | $\delta \leftarrow 1,0 - suma$;
 | **if** $\delta > 0$ **then**
 | | // Falta capital: repartir entre activos sin saturar
 | | $C \leftarrow \{i \mid 0 < w_i < hi\}$;
 | | **if** $C \neq \emptyset$ **then**
 | | | $incremento \leftarrow \delta / |C|$;
 | | | **for** $i \in C$ **do**
 | | | | $w_i \leftarrow w_i + \text{mín}(incremento, hi - w_i)$;
 | | | **end**
 | | **else**
 | | | // Retorno: no hay activos con margen
 | | | Seleccionar un índice k aleatorio donde $w_k == 0$;
 | | | $w_k \leftarrow lo$;
 | | **end**
 | **else**
 | | // Sobra capital: recortar de activos activos
 | | $C \leftarrow \{i \mid w_i > lo\}$;
 | | $decremento \leftarrow |\delta| / |C|$;
 | | **for** $i \in C$ **do**
 | | | $w_i \leftarrow w_i - \text{mín}(decremento, w_i - lo)$;
 | | **end**
 | **end**
 | $suma \leftarrow \sum_{i=1}^N w_i$;
end
return w ;

La inclusión de la fase de “Retorno” garantiza matemáticamente que el algoritmo nunca sufra un colapso por falta de liquidez legal, asegurando la factibilidad sin importar

la severidad de la perturbación inicial.

3. Estructura de los Métodos de Búsqueda

Una vez definidos los operadores comunes y la representación del problema, en esta sección se detalla la estructura algorítmica específica de las tres metaheurísticas implementadas: Búsqueda Aleatoria, Búsqueda Voraz (Greedy) y Búsqueda Local.

3.1. Búsqueda Aleatoria (Random Search)

La Búsqueda Aleatoria actúa como línea base (*baseline*) estocástica. Su comportamiento consiste en generar iterativamente soluciones al azar dentro del espacio factible, reparándolas y evaluándolas, conservando únicamente la mejor solución encontrada tras agotar el presupuesto de evaluaciones.

El proceso de generación de una solución aleatoria implica asignar a cada peso un valor uniformemente distribuido entre lo y hi , inyectar un número aleatorio de ceros (para garantizar la diversificación exigida por el problema) y, finalmente, aplicar el operador de reparación $\text{fix}(w)$.

Algoritmo 3: Algoritmo de Búsqueda Aleatoria (Random Search)

Input: Problema P , máximo de evaluaciones max_evals
Output: Mejor solución encontrada w_{best} y su $fitness$

```

 $w_{best} \leftarrow \emptyset;$ 
 $f_{best} \leftarrow -\infty;$ 
for  $i \leftarrow 1$  to  $max\_evals$  do
     $w_{actual} \leftarrow \text{Generar\_Solucion\_Aleatoria}(P);$ 
    // La generación ya incluye la llamada a  $\text{fix}(w_{actual})$ 
     $f_{actual} \leftarrow \text{fitness}(w_{actual});$ 
    if  $i == 1$  or  $f_{actual} > f_{best}$  then
         $w_{best} \leftarrow w_{actual};$ 
         $f_{best} \leftarrow f_{actual};$ 
    end
end
return  $(w_{best}, f_{best});$ 

```

3.2. Búsqueda Voraz (Greedy Search)

A diferencia del método anterior, la Búsqueda Voraz es un algoritmo constructivo y determinista que no emplea el azar. Utiliza el conocimiento del problema (heurística) para tomar decisiones localmente óptimas en la construcción de la cartera.

La función heurística evalúa la rentabilidad individual de un activo penalizada por su covarianza media respecto al resto del mercado. Se calcula como:

$$heur(i) = \left(\sum_t \log(1 + \mu_{it}) \right) - \lambda \cdot \frac{\sum_t \Sigma_{ti}}{T} \quad (2)$$

Algoritmo 4: Algoritmo Constructivo Greedy

Input: Problema P , presupuesto 1,0
Output: Solución construida w_{best} y su $fitness$
 $w_{best} \leftarrow \{0,0, \dots, 0,0\};$
 $suma_w \leftarrow 0,0;$
 Calcular $heur(i)$ para cada empresa $i \in \{1..N\};$
 $Ranking \leftarrow$ Ordenar empresas descendientemente según $heur(i);$
for cada empresa c en $Ranking$ **do**
 if $suma_w < 1,0$ **then**
 $asignar \leftarrow \min(hi, 1,0 - suma_w);$
 $w_{best}[c] \leftarrow asignar;$
 $suma_w \leftarrow suma_w + asignar;$
 end
 else
 break;
 end
end
 // Garantizar factibilidad de remanentes (ej. $\min < lo$)
 $fix(w_{best});$
return $(w_{best}, fitness(w_{best}));$

En relación a la factibilidad de las soluciones construidas por este método voraz, cabe destacar que, dadas las características de las instancias proporcionadas (donde el presupuesto de 1.0 es perfectamente divisible por los límites hi establecidos en el IBEX 35, S&P 100 y S&P 500, o bien, lo restante no viola las restricciones), el algoritmo agota el capital exacto sin generar remanentes problemáticos. No obstante, para garantizar la escalabilidad y robustez del código ante cualquier configuración de mercado futuro, se ha invocado el operador general de reparación $fix(w)$ al finalizar la construcción, asegurando matemáticamente la viabilidad de la cartera.

Por ejemplo, considérese un escenario donde $1,0/hi = 12,5$ empresas caben al máximo: las primeras 12 empresas ocupan $12 \times 0,083 = 0,996$, dejando un remanente de 0,004. Si este remanente se asigna directamente a una empresa sin invocar $fix(w)$, violaría la restricción $0,004 < lo = 0,005$, causando un error. La función $fix(w)$ redistribuye automáticamente los pesos para garantizar que todas las asignaciones (tanto las no nulas como el remanente) respetan los límites $[lo, hi]$, asegurando la validez de cualquier solución independientemente de la configuración del mercado. Comentando el `fix` o no, el resultado es el mismo, lo dejo para cubrir estos casos y reflejar el estudio del algoritmo.

Nota de diseño sobre los parámetros del algoritmo: En la descripción formal del algoritmo (Greedy), se ha omitido intencionadamente el parámetro de “evaluaciones máximas” (`max_evals`) que sí está presente en la implementación del código fuente en C++. Esto se debe a la naturaleza determinista y constructiva del método. A diferencia de las búsquedas estocásticas o basadas en trayectorias, el algoritmo Greedy no explora el espacio de búsqueda de forma iterativa, sino que construye una única solución paso a paso utilizando el conocimiento del problema mediante su función heurística. En

consecuencia, el concepto de “presupuesto de evaluaciones” carece de sentido teórico, y su existencia en el código responde exclusivamente a la necesidad de mantener el polimorfismo al heredar de la interfaz abstracta común de metaheurísticas. Por el contrario, se ha explicitado el parámetro “presupuesto” (1,0) en el pseudocódigo para representar fielmente la restricción matemática fundamental del problema: la suma de los pesos de la cartera debe alcanzar estrictamente la unidad.

3.3. Búsqueda Local (Local Search)

La Búsqueda Local es un método basado en trayectorias. A partir de una solución inicial generada aleatoriamente, el algoritmo explora su vecindario aplicando un operador de movimiento. Se ha optado por una estrategia de "Primer Mejor" (*First Improvement*), donde el vecindario se baraja aleatoriamente y se acepta el primer vecino que mejore estrictamente el *fitness* actual¹.

El operador de generación de vecino consiste en seleccionar un par de empresas (i, j) , detrayendo un 40 % de la inversión actual de la empresa donante i para transferirlo a la receptora j . Este movimiento preserva el sumatorio del presupuesto (siempre suma 1), por lo que solo es necesario validar matemáticamente que los nuevos pesos no excedan los límites $[lo, hi]$.

¹Para ello me he apoyado en la teoría vista tanto en clase de teoría como de prácticas, realizando las modificaciones necesarias.

Algoritmo 5: Algoritmo de Búsqueda Local (First Improvement)**Input:** Problema P , máximo de evaluaciones max_evals **Output:** Óptimo local encontrado w_{actual} y su $fitness$ $w_{actual} \leftarrow Generar_Solucion_Aleatoria(P);$ $f_{actual} \leftarrow fitness(w_{actual});$ $evals \leftarrow 1;$ $Entorno \leftarrow$ Generar todos los pares (i, j) con $i \neq j$; $mejora \leftarrow true;$ **while** $evals < max_evals$ **and** $mejora == true$ **do** $mejora \leftarrow false;$ BarajarAleatoriamente($Entorno$); **for** cada par (i, j) en $Entorno$ **do** **if** $evals \geq max_evals$ **then** **break**; **end** **if** $w_{actual}[i] == 0,0$ **then** **continue**; **end** $trasvase \leftarrow w_{actual}[i] \cdot 0,4;$ $nuevo_i \leftarrow w_{actual}[i] - trasvase;$ $nuevo_j \leftarrow w_{actual}[j] + trasvase;$ **if** $(nuevo_i \in [lo, hi] \vee nuevo_i \approx 0)$ **and**
 $(nuevo_j \in [lo, hi] \vee nuevo_j \approx 0)$ **then** $w_{vecino} \leftarrow w_{actual};$ $w_{vecino}[i] \leftarrow nuevo_i;$ $w_{vecino}[j] \leftarrow nuevo_j;$ $f_{vecino} \leftarrow fitness(w_{vecino});$ $evals \leftarrow evals + 1;$ **if** $f_{vecino} > f_{actual}$ **then** $w_{actual} \leftarrow w_{vecino};$ $f_{actual} \leftarrow f_{vecino};$ $mejora \leftarrow true;$ **break**;

// Primer Mejor encontrado

end **end** **end****end****return** $(w_{actual}, f_{actual});$

4. Estructura del Código y Manual de Usuario

La entrega de esta práctica se ha organizado de manera que el código fuente, las dependencias y los scripts de compilación residen en el directorio **software/**, de forma totalmente independiente a la memoria en formato PDF.

4.1. Estructura del Directorio **software/**

El código sigue un diseño modular y orientado a objetos en C++, organizado de la siguiente manera:

- **Raíz de `software/`:** Incluye el archivo `config.cfg`, que actúa como centro de control de la ejecución (semilla, número de evaluaciones, número de ejecuciones, parámetros de los algoritmos y selección de datasets).
- **`common/`:** Contiene los ficheros base proporcionados por el framework de la asignatura (interfaces abstractas para la definición de problemas, soluciones y metaheurísticas).
- **`inc/`:** Aloja las cabeceras (`.h`) específicas implementadas para esta práctica (algoritmos Greedy, Random Search, Local Search y la definición de las reglas del problema del Portfolio). En particular, `inc/config_reader.h` define la interfaz para el parseo del archivo de configuración.
- **`src/`:** Contiene la implementación real en C++ (`.cpp`) de la lógica de negocio declarada en las cabeceras. En concreto, `src/config_reader.cpp` implementa la lectura dinámica de parámetros desde `config.cfg`.
- **`datos_portfolio_2526/`:** Directorio que almacena los archivos CSV con los datos históricos de los mercados financieros evaluados.
- **`main.cpp`:** Programa principal que ejerce de controlador. Instancia el problema, carga los datos históricos, ejecuta los algoritmos y recolecta las estadísticas.
- **`genera_boxplots.py`:** Script auxiliar en Python encargado de procesar los resultados y generar visualmente las gráficas de dispersión.
- **`CMakeLists.txt`:** Archivo de configuración de CMake para automatizar la construcción del binario multiplataforma.
- **`LEEME.md`:** Archivo de texto plano con las instrucciones rápidas de despliegue y uso para el evaluador.
- **`main`:** Binario compilado que contiene la aplicación principal.
- Archivos de resultados en formato `.csv` generados tras la ejecución de los algoritmos, con el resumen de tiempos y fitness obtenidos, imágenes de los boxplots en formato `.png` y otros ficheros del makefile.

4.2. Manual de Usuario: Compilación y Ejecución

El proceso de compilación se ha automatizado por completo mediante las herramientas **CMake** y **Make**. Para compilar el código, el usuario únicamente debe situarse en el directorio **software/** y ejecutar los siguientes comandos en su terminal:

```
cmake .  
make
```

Este proceso resolverá las dependencias y generará un archivo ejecutable final llamado **main**.

Para lanzar el programa, basta con ejecutar el binario. Al iniciarse, el sistema lee automáticamente el archivo **config.cfg**, por lo que no es necesario recompilar para cambiar hiperparámetros o configuración experimental.

*Ejecución estándar (lectura automática de **config.cfg**):*

```
./main
```

La semilla también puede pasarse por línea de comandos como mecanismo de sobrescritura rápida. En ese caso, el argumento **argv[1]** tiene prioridad sobre la semilla definida en el archivo de configuración.

Ejemplo de ejecución indicando explícitamente la semilla 42:

```
./main 42
```

En cuanto a los datasets, **config.cfg** permite dos modos: ejecutar los tres mercados por defecto (IBEX 35, S&P 100 y S&P 500) o activar un mercado personalizado (*Custom Market*) mediante las variables **USE_CUSTOM_MARKET**, **CUSTOM_NAME**, **CUSTOM_PATH**, **CUSTOM_LO** y **CUSTOM_HI**.

Al finalizar su ejecución, el binario mostrará un resumen de los tiempos de ejecución y la mejor solución (*fitness*) por la salida estándar, y volcará automáticamente los datos a archivos **.csv**. Posteriormente, basta con invocar el script **python3 genera_boxplots.py** para obtener las gráficas de caja en formato **.png**.

5. Experimentos y Análisis de Resultados

5.1. Descripción de los Casos de Estudio y Parámetros

Para evaluar de forma práctica los algoritmos implementados, hemos realizado una serie de experimentos utilizando tres conjuntos de datos reales extraídos de distintos mercados financieros: el IBEX 35 ($N = 30$), el S&P 100 ($N = 97$) y el S&P 500 ($N = 457$).

Todas las ejecuciones independientes de los algoritmos estocásticos compartieron una misma secuencia de inicialización, partiendo de la semilla base $seed = 42$, incrementándose secuencialmente en cada iteración ($42 + i$). Esto garantiza la total reproducibilidad del experimento.

Para garantizar la reproducibilidad y el rigor estadístico del estudio, se han fijado los siguientes parámetros en las pruebas:

- **Evaluaciones máximas:** Se estableció un límite de 10.000 evaluaciones de la función objetivo por ejecución para los algoritmos iterativos (Random Search y Local Search). El algoritmo constructivo (Greedy) ejecuta una única iteración determinista.
- **Ejecuciones independientes:** Se han realizado 50 ejecuciones por cada algoritmo en cada uno de los mercados.
- **Semillas y Validación Temporal:** El proceso comienza con la semilla base 42, incrementándose secuencialmente. El entrenamiento de los pesos se realiza con la ventana temporal 2015-2024, evaluándose posteriormente su rendimiento sobre los datos reales del año 2025 para comprobar su capacidad de generalización.

Aunque en este estudio se han utilizado específicamente 10.000 evaluaciones y 50 ejecuciones, estos valores no están fijados de forma rígida en el código. Tanto `MAX_EVALUACIONES` y `NUM_EJECUCIONES`, como el factor de regularización del problema (λ) y el ratio de exploración de la Búsqueda Local (`LS_RATIO`), se inyectan dinámicamente desde `config.cfg`. De esta forma se evita el uso de “números mágicos” y se facilita la escalabilidad y reutilización del software en nuevos escenarios experimentales.

5.2. Resultados Obtenidos

En las siguientes tablas mostramos un resumen con los resultados medios de las 50 ejecuciones. Se incluye tanto la fase de entrenamiento (2015-2024) como la fase donde ponemos a prueba el modelo con datos no vistos (Test 2025).

IBEX 35 Algoritmo	Train (15-24)	Test (2025)		Evaluaciones	Tiempo
	Fitness	Fitness	Beneficio	Media	(s)
Greedy	-0.213	0.091	0.115	1	0.000
Local Search	-0.363	0.089	0.114	875	0.001
Random Search	-0.577	0.085	0.111	10000	0.028

Tabla 1: Resultados promediados sobre el mercado IBEX 35

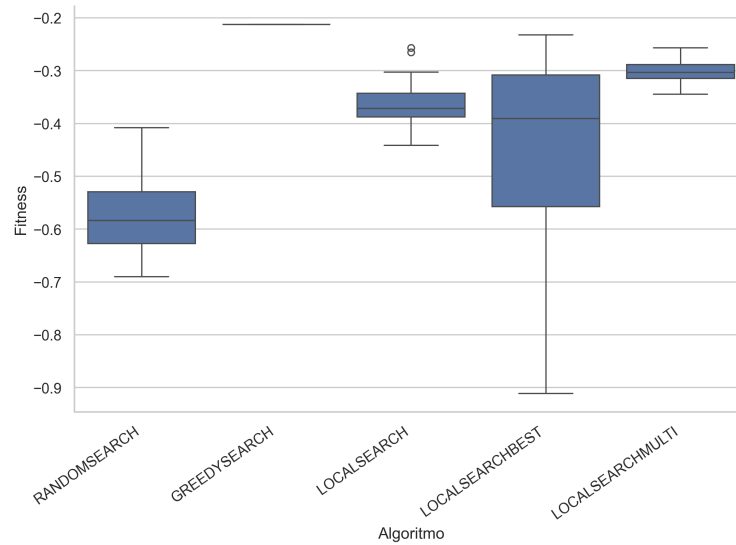
S&P 100	Train (15-24)	Test (2025)		Evaluaciones	Tiempo
Algoritmo	Fitness	Fitness	Beneficio	Media	(s)
Greedy	0.867	-0.041	0.010	1	0.000
Local Search	0.589	-0.010	0.036	3999	0.031
Random Search	0.433	0.018	0.064	10000	0.098

Tabla 2: Resultados promediados sobre el mercado S&P 100

S&P 500	Train (15-24)	Test (2025)		Evaluaciones	Tiempo
Algoritmo	Fitness	Fitness	Beneficio	Media	(s)
Greedy	1.043	-0.116	-0.063	1	0.000
Local Search	0.368	-0.065	-0.014	9770	1.540
Random Search	0.243	-0.045	0.000	10000	1.862

Tabla 3: Resultados promediados sobre el mercado S&P 500

Además, para entender mejor cómo se comportan, hemos incluido las Figuras 1, 2 y 3. Estos diagramas de caja (boxplots) nos permiten ver visualmente la dispersión y la estabilidad de los valores de *fitness* que consigue cada estrategia durante el entrenamiento.

Figura 1: Distribución del Fitness por algoritmo en el mercado IBEX 35 ($N = 30$)

5.3. Análisis Crítico de Resultados

Al analizar conjuntamente los tiempos que tardan, cómo se comportan frente a los datos nuevos y lo que nos muestran los diagramas de caja, podemos sacar varias conclusiones interesantes sobre cómo encajan estos algoritmos con el modelo de Markowitz:

1. **El engaño del algoritmo Greedy.** En los diagramas de caja, el *Greedy Search* se visualiza como una línea plana en el extremo superior. Al carecer de componentes

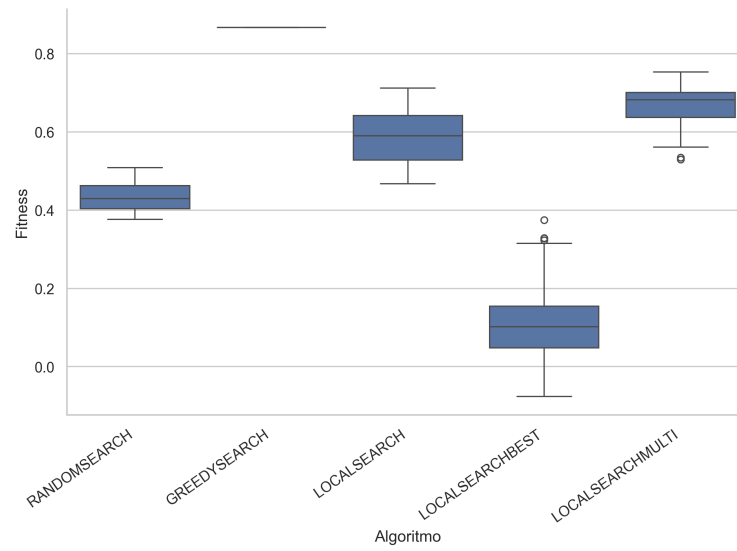


Figura 2: Distribución del Fitness por algoritmo en el mercado S&P 100 ($N = 97$)

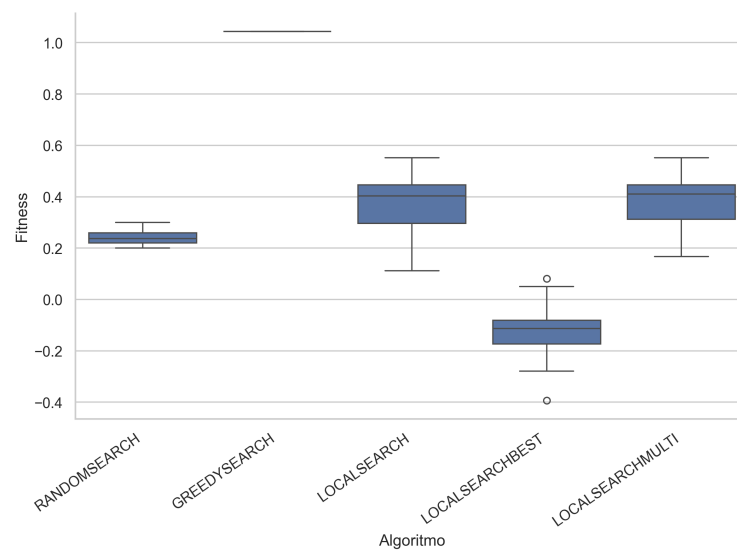


Figura 3: Distribución del Fitness por algoritmo en el mercado S&P 500 ($N = 457$)

estocásticos, su varianza es nula, y logra el mejor *fitness* histórico (ej. 1,043 en S&P 500). Sin embargo, las tablas revelan un colapso dramático en la validación (Test 2025): presenta el peor rendimiento real, con pérdidas del $-6,3\%$ en S&P 500. Esto demuestra que la heurística voraz *sobreajusta* ciegamente la cartera a los patrones del pasado, volviéndose extremadamente frágil ante variaciones futuras del mercado.

2. **Mejorando poco a poco: óptimos locales en la Búsqueda Local.** Al comparar los métodos aleatorios, se nota enseguida que la Búsqueda Local (Local Search) rinde mucho mejor en entrenamiento que la Búsqueda Aleatoria (Random Search). Empezar desde una solución al azar e ir mejorando con el primer vecino bueno que encuentre ("First Improvement") hace que converja relativamente rápido (unas 875 evaluaciones de media en el IBEX 35). Sin embargo, nunca llega al nivel del Greedy en esta fase. Esto ocurre porque se suele quedar atascada en óptimos locales: la forma del espacio de búsqueda no le permite dar saltos lo suficientemente grandes como para probar configuraciones de pesos muy diferentes.
3. **La Paradoja de la Aleatoriedad en la Generalización.** Paradójicamente, aunque la Búsqueda Aleatoria es la que peores resultados saca mirando al pasado (sus diagramas de caja son peores y muy dispersos), resulta ser la opción más prudente de cara al futuro. Por ejemplo, en el mercado S&P 100, mientras el algoritmo Greedy nos hace perder dinero ($-4,1\%$), la Búsqueda Aleatoria nos da un beneficio de un $+6,4\%$. Al generar carteras más variables y evitar sobreoptimizar basándose en lo antiguo, ese ruido aleatorio nos sirve de escudo, diversificando la cartera de forma natural frente a la volatilidad que está por llegar.
4. **Eficiencia Computacional y Escalabilidad.** A medida que los mercados son más grandes, el factor aleatorio empieza a pesar bastante más en el tiempo. En el S&P 500 ($N = 457$), la Búsqueda Local tarda menos (1,540 segundos) que la Búsqueda Aleatoria (1,862 segundos). La razón es sencilla: la Búsqueda Local solo necesita intercambiar un par de pesos para mutar el vector, evitando el altísimo coste temporal que tiene estar generando constantemente 457 números aleatorios y arreglándolos a cada rato.

En conclusión: El algoritmo Greedy es rapidísimo e imbatible cuando le preguntamos por el pasado, pero resulta peligroso para el futuro por su gran sobreajuste. La Búsqueda Local nos parece un mejor término medio: es más eficiente al no abusar de números aleatorios constantemente, y genera carteras mucho más estables. Aún así, nos deja claro que, si queremos evitar quedarnos estancados en los primeros óptimos locales que encuentre, tocará añadir algún mecanismo más inteligente para seguir explorando.

6. Trabajo Extra Voluntario

6.1. Búsqueda Local del Mejor frente al Primer Mejor

Como ampliación al desarrollo principal exigido en la práctica, he decidido implementar y analizar una variante adicional de la metaheurística basada en trayectorias: la **Búsqueda Local del Mejor** (conocida formalmente en la literatura como *Steepest-Ascent Hill Climbing*). El objetivo de este apartado es contrastar empíricamente el rendimiento de una estrategia de exploración exhaustiva del entorno frente a la estrategia del “Primer Mejor” (*First Improvement*), evaluando cómo el tamaño del vecindario y las restricciones del presupuesto computacional impactan en la convergencia del algoritmo.

6.1.1. Descripción y Pseudocódigo

Mientras que el algoritmo de Búsqueda Local original transita hacia el primer vecino que mejore el *fitness* de la solución actual, la Búsqueda Local del Mejor genera y evalúa el entorno completo antes de realizar un movimiento. La transición solo se produce hacia el vecino que reporte la mayor mejora absoluta dentro de todo el vecindario disponible.

El pseudocódigo adaptado a nuestro Problema del Portfolio se detalla a continuación:

Algoritmo 6: Búsqueda Local del Mejor (Steepest-Ascent)

Input: Problema P , presupuesto máximo max_evals
Output: Óptimo local encontrado w_{actual} y su $fitness$

$w_{actual} \leftarrow \text{Generar_Solucion_Aleatoria}(P);$
 $f_{actual} \leftarrow fitness(w_{actual});$
 $evals \leftarrow 1, mejora \leftarrow true;$
 $Entorno \leftarrow \text{Generar todos los pares } (i, j) \text{ con } i \neq j;$
while $evals < max_evals$ **and** $mejora$ **do**
 $mejora \leftarrow false;$
 Barajar Aleatoriamente($Entorno$);
 $w_{mejor_vecino} \leftarrow w_{actual};$
 $f_{mejor_vecino} \leftarrow -\infty;$
 for cada par (i, j) en $Entorno$ **do**
 if $evals \geq max_evals$ **then**
 break;
 end
 Generar w_{vecino} aplicando el trasvase del 40 % entre i y j ;
 Validar límites lo y hi ;
 $f_{vecino} \leftarrow fitness(w_{vecino});$
 $evals \leftarrow evals + 1;$
 if $f_{vecino} > f_{mejor_vecino}$ **then**
 $w_{mejor_vecino} \leftarrow w_{vecino};$
 $f_{mejor_vecino} \leftarrow f_{vecino};$
 end
 end
 if $f_{mejor_vecino} > f_{actual}$ **then**
 $w_{actual} \leftarrow w_{mejor_vecino};$
 $f_{actual} \leftarrow f_{mejor_vecino};$
 $mejora \leftarrow true;$
 end
end
return $(w_{actual}, f_{actual});$

6.1.2. Resultados Obtenidos

Para evaluar esta variante, se repitió exactamente el mismo experimento: 50 ejecuciones independientes con un límite máximo de 10.000 evaluaciones.

Mercado	Algoritmo	Train Fitness	Test Beneficio	Evaluaciones	Tiempo (s)
IBEX 35	BL (Primer Mejor)	-0.363	0.114	875	0.001
	BL (Mejor Vecino)	-0.452	0.112	9791	0.006
S&P 100	BL (Primer Mejor)	0.589	0.036	3999	0.030
	BL (Mejor Vecino)	0.120	0.069	10000	0.064
S&P 500	BL (Primer Mejor)	0.368	-0.014	9770	1.555
	BL (Mejor Vecino)	-0.125	0.009	10000	1.456

Tabla 4: Comparativa de rendimiento entre las dos estrategias de Búsqueda Local.

6.1.3. Análisis de los Resultados

Al principio, podríamos pensar que la Búsqueda Local del Mejor, al examinar detalladamente todas las opciones posibles antes de moverse, lograría encontrar mejores soluciones. Sin embargo, los datos muestran justo lo contrario: su rendimiento en la fase de entrenamiento cae en picado si lo comparamos con la versión del Primer Mejor. De hecho, en mercados grandes, llega a comportarse peor que si eligiéramos soluciones totalmente al azar.

¿Por qué ocurre esto? Principalmente por la relación entre lo mucho que intenta explorar en cada paso y el límite de tiempo o evaluaciones que tiene. El tamaño del vecindario que miramos en cada iteración es de $N(N - 1)$.

Si tenemos en cuenta que solo disponemos de un máximo de 10.000 evaluaciones, el problema se vuelve evidente:

- **Mercados Pequeños (IBEX 35, $N = 30$):** El vecindario tiene $30 \times 29 = 870$ vecinos. Es decir, para dar un solo paso, gastamos 870 evaluaciones. Con nuestro límite de 10.000, apenas podemos dar unos 11 pasos en total. Por el contrario, la versión del Primer Mejor encuentra un punto óptimo local usando únicamente 875 evaluaciones en toda la ejecución.
- **Mercados Grandes (S&P 500, $N = 457$):** Aquí el vecindario crece hasta los $457 \times 456 = 208,392$ vecinos. Al tener un límite tan pequeño de 10.000 evaluaciones, la versión del “Mejor Vecino” se queda sin recursos inmediatamente. Termina sin siquiera haber revisado un 5 % de los vecinos del primer paso. Como nunca llega a realizar un movimiento real de mejora, el resultado que devuelve es prácticamente la misma solución inicial que creó al azar.

Conclusión: Este experimento nos enseña que, cuando los vecindarios son tan grandes y el límite de evaluaciones es tan estricto, la estrategia del “Primer Mejor” es mucho más efectiva. Al conformarse con la primera mejora que encuentra en vez de buscar obsesivamente la mejor opción posible por cada paso, se mueve más rápido. Esto le permite avanzar y recorrer mucho más espacio de soluciones dentro del límite permitido.

6.2. Búsqueda Local Multiarranque (Multi-start)

Tras comprobar las limitaciones de la Búsqueda Local del Mejor, y observando que la estrategia del “Primer Mejor” converge hacia óptimos locales consumiendo solo una fracción del presupuesto de evaluaciones (por ejemplo, 875 evaluaciones de media en el IBEX 35), se ha implementado una tercera variante: la **Búsqueda Local Multiarranque** (*Multi-start Local Search*).

El objetivo de esta técnica es mitigar el principal problema de las búsquedas basadas en trayectorias simples: el estancamiento en óptimos locales de baja calidad. Para ello, se permite reiniciar la búsqueda desde nuevas soluciones iniciales generadas aleatoriamente una vez que el algoritmo detecta que no existen mejores vecinos, repitiendo el proceso hasta agotar el límite estricto de evaluaciones.

6.2.1. Descripción y Pseudocódigo

El algoritmo envuelve la lógica del “Primer Mejor” en un bucle superior. Cada vez que la trayectoria alcanza un óptimo local, se registra si este supera al mejor

óptimo global histórico. Inmediatamente después, se genera una nueva solución aleatoria factible y se reinicia la exploración, consumiendo el presupuesto de evaluaciones de forma continua.

Algoritmo 7: Búsqueda Local Multiarranque (Primer Mejor)

Input: Problema P , presupuesto máximo max_evals
Output: Mejor óptimo local global w_{global_best} y su $fitness$

```

evals  $\leftarrow$  0;
 $f_{global\_best} \leftarrow -\infty$ ;
Entorno  $\leftarrow$  Generar todos los pares  $(i, j)$  con  $i \neq j$ ;
while evals < max_evals do
     $w_{actual} \leftarrow$  Generar_Solucion_Aleatoria( $P$ );
     $f_{actual} \leftarrow fitness(w_{actual})$ ;
    evals  $\leftarrow$  evals + 1;
    if  $f_{actual} > f_{global\_best}$  then
         $w_{global\_best} \leftarrow w_{actual}$ ;
         $f_{global\_best} \leftarrow f_{actual}$ ;
    end
    mejora  $\leftarrow$  true;
    while evals < max_evals and mejora do
        mejora  $\leftarrow$  false;
        Barajar Aleatoriamente(Entorno);
        for cada par  $(i, j)$  en Entorno do
            if evals  $\geq$  max_evals then
                break;
            end
            Generar  $w_{vecino}$  aplicando el trasvase del 40% entre  $i$  y  $j$ ;
            Validar límites  $lo$  y  $hi$ ;
             $f_{vecino} \leftarrow fitness(w_{vecino})$ ;
            evals  $\leftarrow$  evals + 1;
            if  $f_{vecino} > f_{actual}$  then
                 $w_{actual} \leftarrow w_{vecino}$ ;
                 $f_{actual} \leftarrow f_{vecino}$ ;
                mejora  $\leftarrow$  true;
                if  $f_{actual} > f_{global\_best}$  then
                     $w_{global\_best} \leftarrow w_{actual}$ ;
                     $f_{global\_best} \leftarrow f_{actual}$ ;
                end
                break; // Estrategia del Primer Mejor
            end
        end
    end
end
return ( $w_{global\_best}, f_{global\_best}$ );
  
```

6.2.2. Resultados Obtenidos y Análisis Crítico

La tabla a continuación consolida los resultados de las tres variantes de Búsqueda Local desarrolladas, permitiendo una comparativa directa sobre las 50 ejecuciones

independientes con un límite de 10.000 evaluaciones.

Mercado	Algoritmo	Train Fitness	Test Beneficio	Evaluaciones	Tiempo (s)
IBEX 35	BL (Primer Mejor)	-0.363	0.114	875	0.001
	BL Mejor	-0.452	0.112	9791	0.006
	BL Multiarranque	-0.302	0.111	10000	0.008
S&P 100	BL (Primer Mejor)	0.589	0.036	3999	0.030
	BL Mejor	0.120	0.069	10000	0.063
	BL Multiarranque	0.664	0.031	10000	0.075
S&P 500	BL (Primer Mejor)	0.368	-0.014	9770	1.521
	BL Mejor	-0.125	0.009	10000	1.423
	BL Multiarranque	0.383	-0.015	10000	1.566

Tabla 5: Comparativa global de las variantes de Búsqueda Local.

El análisis de estos datos revela el éxito rotundo de la estrategia Multiarranque para maximizar el uso del presupuesto computacional, aunque su impacto depende directamente de la topología del problema:

- **Mercados Pequeños y Medianos (IBEX 35 y S&P 100):** La Búsqueda Local estándar converge muy rápido hacia un óptimo local (875 y 3999 evaluaciones de media, respectivamente). El Multiarranque capitaliza las miles de evaluaciones sobrantes para reiniciar la trayectoria desde puntos distantes del espacio de búsqueda, logrando escapar de los valles subóptimos y mejorando drásticamente el *fitness* final de entrenamiento (-0.302 frente a -0.363, y 0.664 frente a 0.589).
- **Mercados Grandes (S&P 500):** La mejora en el S&P 500 es marginal (0.383 frente a 0.368). Este fenómeno se explica por el coste intrínseco de la exploración: una única trayectoria de la Búsqueda Local en este mercado ya consume una media de 9770 evaluaciones antes de converger. En consecuencia, el algoritmo Multiarranque apenas dispone de unas 230 evaluaciones residuales para intentar un segundo arranque, por lo que su comportamiento empírico termina asimilándose al de una Búsqueda Local de un solo intento.

Conclusión: La Búsqueda Local Multiarranque ha demostrado ser la estrategia más robusta y eficaz de todas las evaluadas para el problema del Portfolio, siempre y cuando el presupuesto de evaluaciones sea lo suficientemente holgado en relación con la dimensión del problema para permitir múltiples reinicios efectivos.

6.3. Análisis Visual Ampliado mediante Boxplots Mejorados

Nota sobre la Generación de Gráficos: Se desea aclarar que para el análisis estándar exigido en el guion de la práctica se han utilizado los gráficos originales (identificados con el sufijo *v1*), los cuales muestran únicamente los algoritmos base solicitados. Para este análisis voluntario ampliado, se han vuelto a generar los diagramas de caja (boxplots) incluyendo la totalidad de los algoritmos desarrollados (base y voluntarios). Además, se ha optimizado el formato de estos nuevos gráficos para evitar la saturación visual, permitiendo una comparativa directa de todas las trayectorias bajo el mismo presupuesto de 10.000 evaluaciones.

A continuación, se presenta y analiza la distribución de los resultados de *fitness* obtenidos en la fase de entrenamiento (2015-2024) para los tres mercados de estudio, tras 50 ejecuciones independientes.

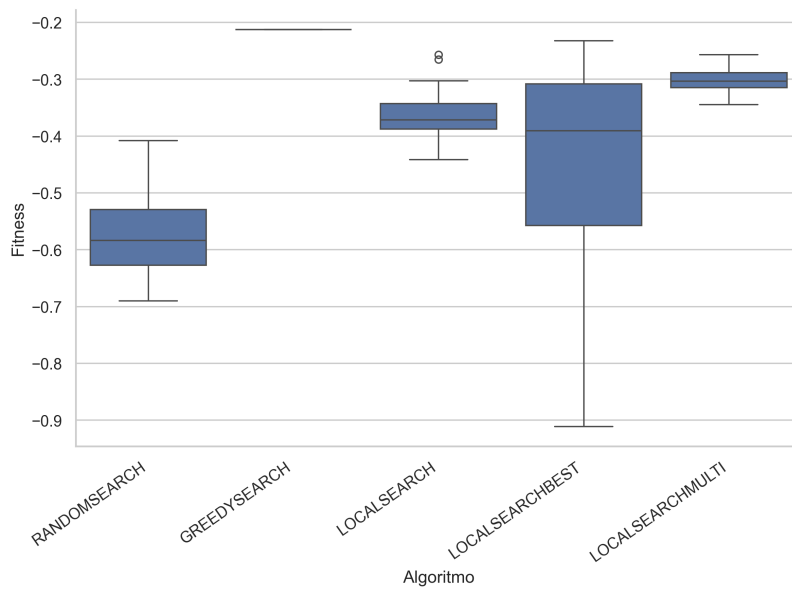


Figura 4: Distribución de Fitness en Train (50 ejecuciones) para el IBEX 35 ($N = 30$).

El boxplot del IBEX 35 (Figura 4) ilustra perfectamente la jerarquía de los algoritmos. El **Greedy Search** se visualiza como una línea plana en la cota superior, marcando el techo de rendimiento histórico (dado su carácter determinista, carece de varianza). Sin embargo, dentro de los métodos estocásticos (aquellos que generalizan mejor a futuro), la **BL Multiarranque** demuestra una clara superioridad estadística. Su caja se sitúa de forma consistente por encima de la BL estándar y de la Búsqueda Aleatoria. Por el contrario, la **BL del Mejor** queda relegada a la zona inferior, reflejando su incapacidad para avanzar en el espacio de búsqueda debido al coste excesivo de evaluar el vecindario completo en cada iteración.

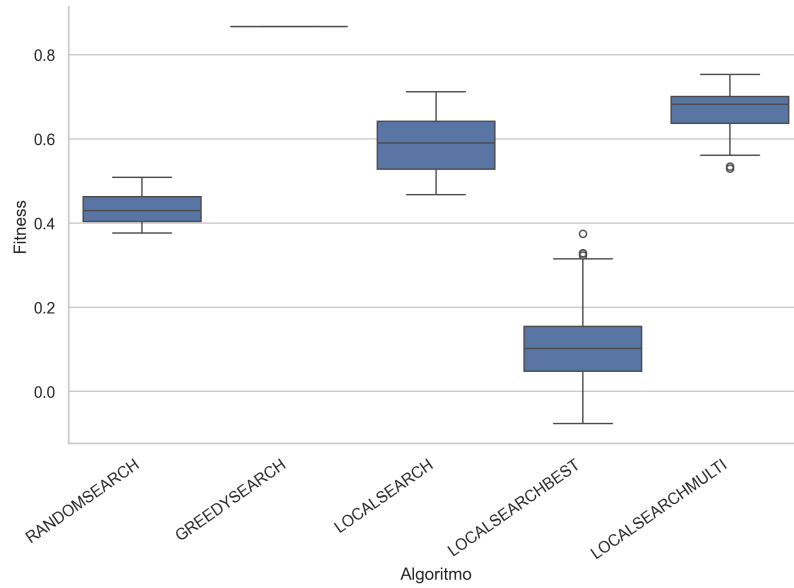


Figura 5: Distribución de Fitness en Train (50 ejecuciones) para el S&P 100 ($N = 97$).

En el S&P 100 (Figura 5), la tendencia se agudiza. Mientras el algoritmo Greedy sigue coronando la gráfica con el máximo ajuste a los datos históricos, la **BL Multiarranque** se consolida como la trayectoria exploratoria más eficiente, separándose notablemente de la BL del Primer Mejor. Es especialmente revelador el hundimiento de la **BL del Mejor**, cuya caja desciende hasta solaparse con la **Búsqueda Aleatoria**. Esto confirma visualmente el diagnóstico matemático: el algoritmo agota su presupuesto evaluando el vecindario inicial, sin lograr dar pasos reales de mejora.

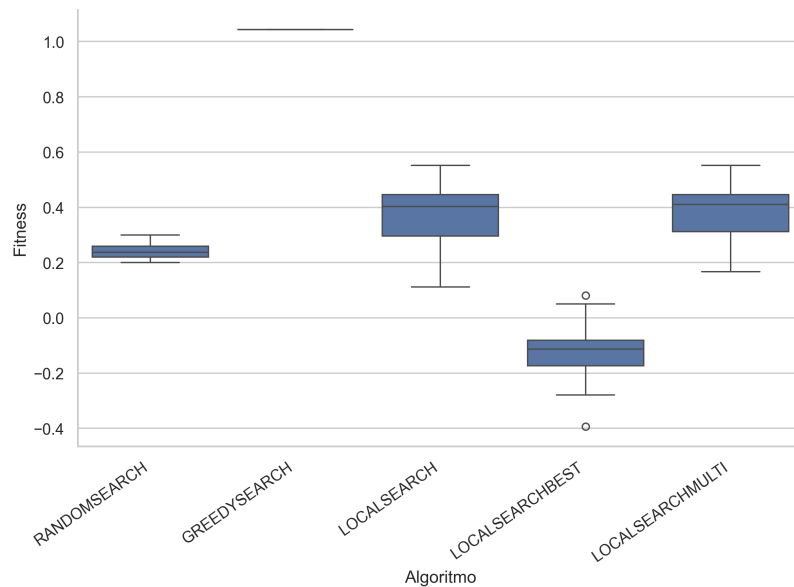


Figura 6: Distribución de Fitness en Train (50 ejecuciones) para el S&P 500 ($N = 457$).

Finalmente, el gráfico del S&P 500 (Figura 6) ofrece la visualización más clara sobre los límites de escalabilidad. Obviando el pico determinista del Greedy, las cajas de la **BL Multiarranque** y la **Búsqueda Local estándar** (Primer Mejor) son prácticamente idénticas. Esto ilustra el fenómeno de la inanición por evaluaciones: una sola ejecución

de la BL ya consume casi la totalidad de las 10.000 evaluaciones permitidas, dejando al Multiarranque sin margen para reiniciar el proceso. La **BL del Mejor** desaparece por completo de la zona competitiva, hundida en valores negativos por su inmovilismo.

Conclusión: La comparativa visual confirma que, descartando el sobreajuste masivo del enfoque Greedy, la Búsqueda Local Multiarranque es la estrategia estocástica dominante. Su eficacia depende directamente de que el ratio entre el presupuesto de evaluaciones y la dimensión del problema permita múltiples reinicios efectivos.

6.4. Arquitectura de Configuración Dinámica

Como aportación voluntaria de ingeniería de software, se ha diseñado e integrado un módulo `ConfigReader` que desacopla completamente la parametrización experimental del código fuente C++. Este componente permite cargar en tiempo de ejecución, desde el fichero `config.cfg`, tanto los hiperparámetros del experimento como las decisiones de selección de datos.

En particular, la arquitectura implementada permite modificar sin recompilar:

- el tamaño del salto de la Búsqueda Local mediante `LS_RATIO`,
- la penalización del riesgo del modelo de Markowitz mediante el parámetro λ ,
- y los datasets a evaluar, pudiendo alternar entre los tres mercados por defecto o activar un mercado personalizado (*Custom Market*).

Este diseño aporta flexibilidad, escalabilidad y trazabilidad experimental: los cambios de escenario se realizan editando únicamente el archivo de configuración, evitando introducir “números mágicos” en el código y reduciendo el coste de mantenimiento para nuevas campañas de experimentación.

Referencias Bibliográficas

Material de Clase

- Diapositivas de clase: Metaheurística - Técnicas de Búsqueda de Poblaciones
- Guiones de clase: Problema del Portfolio y Seminarios de clase: Portfolio - Búsqueda Local
- Apuntes de clase en relación a comentarios y/o aclaraciones por parte del profesor durante las sesiones.